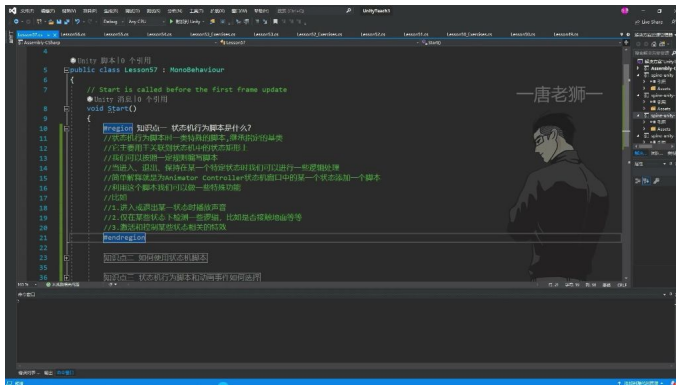


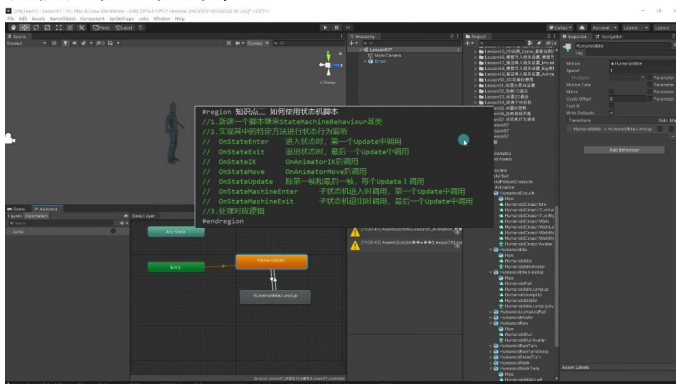
一、状态机行为脚本 00:04

1. 状态机行为脚本是什么 00:38

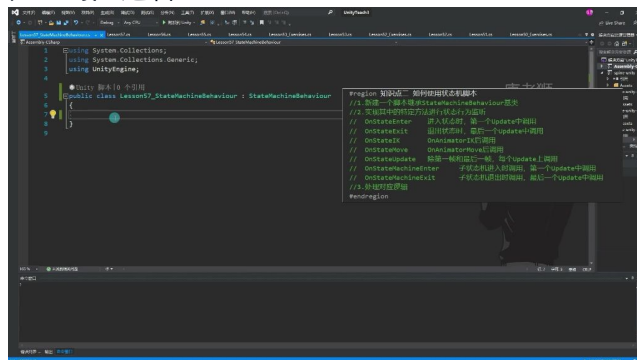


- 定义：一类特殊的脚本，继承自StateMachineBehaviour基类
- 作用：关联到状态机中的状态矩形上，在特定状态时执行逻辑处理
- 功能：
 - 为Animator Controller状态机窗口中的某个状态添加脚本
 - 实现进入/退出/保持状态时的特殊功能
- 典型应用：
 - 进入/退出状态时播放声音
 - 特定状态下检测逻辑（如接触地面检测）
 - 激活和控制状态相关特效

2. 如何使用状态机脚本 03:02

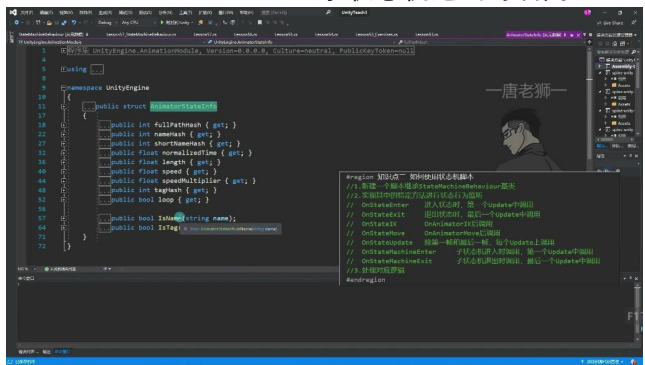


- 实现步骤：
 - 新建脚本继承StateMachineBehaviour基类
 - 重写特定状态行为监听方法
 - 处理对应逻辑



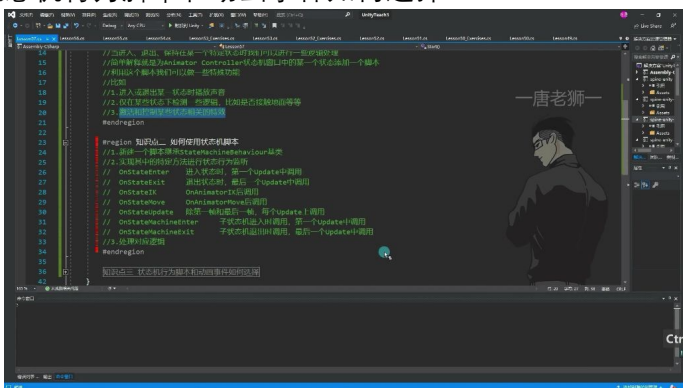
○

- 关键方法：
 - OnStateEnter: 进入状态时第一个Update调用
 - OnStateExit: 退出状态时最后一个Update调用
 - OnStateUpdate: 除首尾帧外每个Update调用
 - OnStateIK: OnAnimatorIK后调用
 - OnStateMove: OnAnimatorMove后调用
 - OnStateMachineEnter: 子状态机进入时调用
 - OnStateMachineExit: 子状态机退出时调用



- 实践技巧：
 - 通过stateInfo.IsName判断当前状态
 - 使用Debug.Log输出调试信息（需用UnityEngine.Debug）
 - 可通过animator参数获取关联对象
 - 脚本可复用，通过public字段配置不同状态行为

3. 状态机行为脚本和动画事件如何选择 13:07



- 选择依据：
 - 状态机行为脚本：功能更强大精确，但使用稍复杂
 - 动画事件：简单易用但功能有限
- 适用场景：
 - 需要精确控制状态转换逻辑时选择状态机行为脚本
 - 简单触发事件时可用动画事件

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
状态机行	继承StateMachineBehaviour基类的特殊脚本，可关联到状态机的特定状态，在进入/退出/保持状态时触发逻辑（如播放音效、检测地面接触）。	与MonoBehaviour的区别：需显式继承基类，无法直接使用Print方法，需通过Debug.Log输出。	★★

为脚本的定义			
状态机脚本的使用步骤	1. 新建脚本继承 <code>StateMachineBehavior</code>; 2. 重写虚方法 (如 <code>OnStateEnter</code>、<code>OnStateUpdate</code>); 3. 通过 <code>AnimatorStateInfo</code> 判断当前状态名; 4. 将脚本添加到状态机的 <code>Add Behavior</code> 中。	关键方法: - <code>OnStateEnter</code>: 状态首次进入时调用; - <code>OnStateExit</code>: 状态退出时调用; - <code>OnStateUpdate</code>: 状态持续时每帧调用。	☆☆
状态机脚本与动画事件的对比	状态机脚本: 更准确 (不受动画过渡影响), 适合状态切换时的逻辑 (如进入/退出动作)。 动画事件: 更灵活, 可在动画任意时刻触发, 但过渡阶段可能失效。	选择依据: - 需精确控制状态切换 → 状态机脚本; - 需在动画中途触发 → 动画事件。	☆☆
实战应用示例	通用脚本设计: - 将状态名、音效名等设为可配置字段; - 通过 <code>Animator</code> 参数获取关联对象; - 示例: 进入“走路”状态时播放对应音效。	易错点: - 未继承基类导致脚本无法添加; - 混淆 <code>OnStateUpdate</code> 与 <code>Update</code> 生命周期。	☆☆